Estruturas de Repetição, Vetores e Matrizes em Java

PARTE 1 — Estruturas de Repetição

Estruturas de repetição (também chamadas de *loops*) permitem executar um bloco de código várias vezes, enquanto uma condição for verdadeira ou por um número determinado de iterações. São essenciais para evitar código repetitivo e processar coleções de dados.

---

1. `for` — Loop com contador

Usado quando você sabe previamente quantas vezes o loop deve executar.

Sintaxe:

```
for (inicialização; condição; incremento) {
    // bloco de código

 }
```

Exemplo:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Volta número: " + i);

 }
```

Como funciona:

1. `int i = 1` — inicializa a variável de controle
2. `i <= 5` — verifica a condição antes de cada iteração
3. `i++` — incrementa após cada execução do bloco
4. Quando a condição for `false`, o loop encerra

Variações comuns:

```
// Contagem regressiva
for (int i = 10; i >= 1; i--) {
    System.out.println(i);
}
```

```
// Incremento de 2 em 2
for (int i = 0; i <= 20; i += 2) {
    System.out.println(i); // 0, 2, 4, 6...
}

// Loop infinito (evite sem um break)
for (;;) {
    // executa indefinidamente

 }
```

---

2. `while` — Loop com condição prévia

Usado quando você não sabe quantas iterações serão necessárias. A condição é verificada antes de executar o bloco.

Sintaxe:

```
while (condição) {
    // bloco de código

 }
```

Exemplo:

```
int numero = 1;

while (numero <= 5) {
    System.out.println("Número: " + numero);
    numero++;

 }
```

Atenção: se a condição nunca se tornar `false`, o programa entra em loop infinito.

Sempre garanta que a variável de controle seja atualizada dentro do loop.

Exemplo prático — validação de entrada:

```
Scanner sc = new Scanner(System.in);
int idade = -1;
```

```
while (idade < 0 || idade > 120) {
    System.out.print("Digite uma idade válida: ");
    idade = sc.nextInt();
}

System.out.println("Idade registrada: " + idade);
```

---

3. `do/while` — Loop com condição posterior

Similar ao `while`, mas o bloco é executado pelo menos uma vez, pois a condição é verificada depois.

Sintaxe:

```
do {
    // bloco de código

} while (condição);
```

Exemplo:

```
int numero = 1;

do {
    System.out.println("Número: " + numero);
    numero++;

} while (numero <= 5);
```

Diferença prática entre `while` e `do/while`:

```
int x = 10;

// while NÃO executa (condição já é false)
while (x < 5) {
    System.out.println("while: " + x);
}

// do/while executa UMA vez mesmo assim
do {
    System.out.println("do/while: " + x); // imprime 10

} while (x < 5);
```

---

4. `for-each` — Loop para coleções

Versão simplificada do `for`, ideal para percorrer arrays e coleções sem precisar de índice.

Sintaxe:

```
for (tipo elemento : colecao) {
    // bloco de código

 }
```

Exemplo:

```
int[] numeros = {10, 20, 30, 40, 50};

for (int n : numeros) {
    System.out.println(n);

 }
```

Limitação: não dá acesso ao índice do elemento. Se precisar do índice, use o `for` tradicional.

---

5. `break` e `continue`

Duas palavras-chave que controlam o fluxo dentro de loops.

`break` — interrompe o loop imediatamente:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) break; // para no 5
    System.out.println(i); // imprime 1, 2, 3, 4

 }
```

`continue` — pula para a próxima iteração:

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue; // pula os pares
```

```
    System.out.println(i); // imprime 1, 3, 5, 7, 9

}
```

---

6. Loops Aninhados

Um loop dentro de outro. Muito usado para trabalhar com matrizes e tabelas.

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i * j + "\t");
    }
    System.out.println();

}
```

Saída:

```
1	2	3
2	4	6
3	6	9
```

Quando usar cada loop?

Situação	Loop recomendado
Número de iterações conhecido	`for`
Condição avaliada antes	`while`
Executar ao menos uma vez	`do/while`
Percorrer array/coleção sem índice	`for-each`
Interromper no meio	`for` ou `while` + `break`

PARTE 2 — Vetores (Arrays Unidimensionais)

O que é um vetor?

Um vetor (ou array unidimensional) é uma estrutura de dados que armazena múltiplos valores do mesmo tipo em posições de memória contíguas, acessadas por um índice que começa em `0`.

Pense em um vetor como uma fileira de caixas numeradas — cada caixa guarda um valor e você acessa cada uma pelo seu número.

```
Índice:  [0]   [1]   [2]   [3]   [4]
Valor:    10    20    30    40    50
```

Declaração e Inicialização

Forma 1 — Declarar e depois atribuir:

```
int[] numeros = new int[5]; // cria vetor com 5 posições (valores padrão =
0)
numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;

 numeros[4] = 50;
```

Forma 2 — Declarar e inicializar junto:

```
 int[] numeros = {10, 20, 30, 40, 50};
```

Forma 3 — Declarar e inicializar com `new`:

```
 int[] numeros = new int[]{10, 20, 30, 40, 50};
```

Outros tipos:

```java
double[] precos = {1.99, 5.50, 12.75};
String[] nomes = {"Ana", "Bruno", "Carlos"};
boolean[] flags = {true, false, true};

 char[] letras = {'a', 'e', 'i', 'o', 'u'};
```

Acessando Elementos

```java
String[] frutas = {"Maçã", "Banana", "Laranja", "Uva"};

System.out.println(frutas[0]); // Maçã
System.out.println(frutas[2]); // Laranja

 System.out.println(frutas[frutas.length - 1]); // Uva (último elemento)
```

`length` retorna o tamanho do vetor. O último índice válido é sempre `length - 1`.

Acessar um índice inexistente lança `ArrayIndexOutOfBoundsException`.

Percorrendo um Vetor

Com `for` tradicional (acesso ao índice):

```java
int[] notas = {8, 7, 9, 6, 10};

for (int i = 0; i < notas.length; i++) {
    System.out.println("Nota[" + i + "] = " + notas[i]);

 }
```

Com `for-each` (mais limpo, sem índice):

```java
int[] notas = {8, 7, 9, 6, 10};

for (int nota : notas) {
    System.out.println("Nota: " + nota);

 }
```

Operações Comuns com Vetores

Soma e Média:

```
int[] valores = {4, 8, 15, 16, 23, 42};
int soma = 0;

for (int v : valores) {
    soma += v;
}

double media = (double) soma / valores.length;
System.out.println("Soma: " + soma);

 System.out.println("Média: " + media);
```

Encontrar o Maior e Menor:

```
int[] nums = {3, 7, 1, 9, 4};
int maior = nums[0];
int menor = nums[0];

for (int n : nums) {
    if (n > maior) maior = n;
    if (n < menor) menor = n;
}

System.out.println("Maior: " + maior); // 9

 System.out.println("Menor: " + menor); // 1
```

Busca Linear:

```
int[] dados = {5, 12, 8, 3, 20};
int busca = 8;
boolean encontrado = false;

for (int i = 0; i < dados.length; i++) {
    if (dados[i] == busca) {
        System.out.println("Encontrado no índice: " + i);
        encontrado = true;
        break;
    }
```

```
}

if (!encontrado) System.out.println("Não encontrado.");
```

Inverter um Vetor:

```
int[] original = {1, 2, 3, 4, 5};
int[] invertido = new int[original.length];

for (int i = 0; i < original.length; i++) {
    invertido[i] = original[original.length - 1 - i];
}

// Exibir invertido
for (int n : invertido) {
    System.out.print(n + " "); // 5 4 3 2 1

}
```

Ordenação com `Arrays.sort()`:

```
import java.util.Arrays;

int[] nums = {5, 3, 8, 1, 9, 2};
Arrays.sort(nums);


System.out.println(Arrays.toString(nums)); // [1, 2, 3, 5, 8, 9]
```

Classe `Arrays` — Utilitários

```
import java.util.Arrays;

int[] v = {5, 1, 3, 2, 4};

Arrays.sort(v);                      // ordena crescente
Arrays.fill(v, 0);                   // preenche tudo com 0
int[] copia = Arrays.copyOf(v, v.length); // copia o vetor

System.out.println(Arrays.toString(v));   // exibe como string
```

Vetor como Parâmetro de Método

```
public static int somar(int[] vetor) {
    int total = 0;
    for (int n : vetor) total += n;
    return total;
}

public static void main(String[] args) {
    int[] numeros = {1, 2, 3, 4, 5};
    System.out.println("Soma: " + somar(numeros)); // 15

 }
```

Em Java, arrays são passados por referência — alterações dentro do método afetam o array original.

PARTE 3 — Matrizes (Arrays Bidimensionais)

O que é uma Matriz?

Uma matriz é um array de duas dimensões — basicamente uma tabela com linhas e colunas. Cada elemento é acessado por dois índices: `[linha][coluna]`, ambos começando em `0`.

```
        col0  col1  col2
linha0: [  1    2    3  ]
linha1: [  4    5    6  ]
linha2: [  7    8    9  ]
```

Declaração e Inicialização

Forma 1 — Declarar e atribuir depois:

```
int[][] matriz = new int[3][3]; // 3 linhas, 3 colunas
matriz[0][0] = 1;
matriz[0][1] = 2;
```

```
// ...
```

Forma 2 — Inicialização direta:

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}

};
```

Outros tipos:

```
double[][] precos = new double[4][5];

String[][] tabela = new String[3][3];
```

Acessando Elementos

```
int[][] m = {
    {10, 20, 30},
    {40, 50, 60},
    {70, 80, 90}
};

System.out.println(m[0][0]); // 10
System.out.println(m[1][2]); // 60

System.out.println(m[2][1]); // 80
```

Percorrendo uma Matriz

Com `for` aninhado (recomendado para matrizes):

```
int[][] m = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

```
for (int i = 0; i < m.length; i++) {          // linhas
    for (int j = 0; j < m[i].length; j++) {   // colunas
        System.out.print(m[i][j] + "\t");
    }
    System.out.println();

}
```

Saída:

```
1    2    3
4    5    6
7    8    9
```

Com `for-each` aninhado:

```
for (int[] linha : m) {
    for (int elemento : linha) {
        System.out.print(elemento + "\t");
    }
    System.out.println();

}
```

Dimensões da Matriz

```
int[][] m = new int[4][6];

System.out.println(m.length);     // 4 — número de linhas

 System.out.println(m[0].length);  // 6 — número de colunas
```

Operações Comuns com Matrizes

Soma de todos os elementos:

```
int[][] m = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
```

```
};

int soma = 0;
for (int[] linha : m) {
    for (int val : linha) {
        soma += val;
    }
}

 System.out.println("Soma total: " + soma); // 45
```

Diagonal principal (i == j):

```
int[][] m = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.print("Diagonal principal: ");
for (int i = 0; i < m.length; i++) {
    System.out.print(m[i][i] + " "); // 1 5 9

 }
```

Transposta de uma Matriz:

```
int[][] original = {
    {1, 2, 3},
    {4, 5, 6}
};

int linhas = original.length;
int colunas = original[0].length;
int[][] transposta = new int[colunas][linhas];

for (int i = 0; i < linhas; i++) {
    for (int j = 0; j < colunas; j++) {
        transposta[j][i] = original[i][j];
    }

 }
```

Multiplicação de Matrizes:

```
int[][] A = {{1, 2}, {3, 4}};
```

```
int[][] B = {{5, 6}, {7, 8}};
int[][] C = new int[2][2];

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        for (int k = 0; k < 2; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }

}
```

Matriz Irregular (Jagged Array)

Em Java, é possível criar matrizes onde cada linha tem um número diferente de colunas.

```
int[][] irregular = new int[3][];
irregular[0] = new int[2]; // linha 0 tem 2 colunas
irregular[1] = new int[4]; // linha 1 tem 4 colunas

irregular[2] = new int[1]; // linha 2 tem 1 coluna
```

Arrays Multidimensionais (3D+)

Java suporta arrays com mais de duas dimensões, embora sejam menos comuns.

```
int[][][] cubo = new int[3][3][3]; // array tridimensional

cubo[0][1][2] = 42;
```

Resumo Visual — Diferenças

Característica	Vetor	Matriz
Dimensões	1D	2D (ou mais)
Declaração	`int[] v`	`int[][] m`

Acesso	`v[i]`	`m[i][j]`
Tamanho	`v.length`	`m.length` × `m[0].length`
Loop típico	`for` simples	`for` aninhado
Uso comum	Lista de valores	Tabelas, grades, imagens

Chamando Classes Externas em Java

Sim, Devisate! Isso é totalmente possível em Java, e existem várias formas de fazer isso dependendo do seu contexto. Vou explicar cada abordagem de forma clara e prática em Markdown.

Como o Java Funciona (contexto importante)

Antes de tudo, é essencial entender que o Java não executa arquivos `.java` diretamente em tempo de execução — ele executa bytecode (arquivos `.class`). Então quando dizemos "chamar uma classe de outra", estamos falando de uma classe compilada chamando outra classe compilada.

```
Java1.java  →  javac  →  Java1.class  ┐
                                      ├→ JVM executa tudo

 Java2.java  →  javac  →  Java2.class  ┘
```

Situação 1 — Mesma pasta / mesmo projeto (mais comum)

Essa é a forma mais simples e direta. Se `Java1.java` e `Java2.java` estão no mesmo diretório ou projeto, basta instanciar a classe normalmente.

`Java2.java`:

```java
public class Java2 {

    public void executar() {
        System.out.println("Olá! Eu sou a Java2 sendo executada!");
    }

    public int somar(int a, int b) {
        return a + b;
    }

}
```

`Java1.java` (chama `Java2`):

```java
public class Java1 {
```

```java
    public static void main(String[] args) {
        // Instancia a classe Java2
        Java2 obj = new Java2();

        // Chama o método da Java2
        obj.executar();

        // Chama outro método e usa o retorno
        int resultado = obj.somar(5, 3);
        System.out.println("Resultado da soma: " + resultado);
    }

}
```

Compilando e executando no terminal:

```
# Compilar as duas classes
javac Java1.java Java2.java

# Ou compilar todas de uma vez
javac *.java

# Executar (apenas a classe que tem o main principal)

java Java1
```

Saída:

```
Olá! Eu sou a Java2 sendo executada!

Resultado da soma: 8
```

Situação 2 — Classes em pacotes diferentes

Quando as classes estão organizadas em pacotes (`package`), você precisa usar `import`.

Estrutura de pastas:

```
projeto/
├── pacoteA/
│   └── Java1.java
└── pacoteB/
```

```
    └── Java2.java
```

`Java2.java` (pacote B):

```java
package pacoteB;

public class Java2 {

    public void executar() {
        System.out.println("Java2 do pacoteB executando!");
    }

}
```

`Java1.java` (pacote A, importa `Java2`):

```java
package pacoteA;

import pacoteB.Java2; // importa a classe do outro pacotepublic class Java1
{

    public static void main(String[] args) {
        Java2 obj = new Java2();
        obj.executar();
    }

}
```

Compilando com pacotes:

```
# A partir da pasta raiz do projeto
javac pacoteA/Java1.java pacoteB/Java2.java

# Executar (informando o pacote completo)

java pacoteA.Java1
```

Situação 3 — Chamando o `main` de outra classe diretamente

Se você quiser chamar especificamente o método `main` de `Java2` a partir de `Java1`, é direto assim:

```java
public class Java2 {
```

```
    public static void main(String[] args) {
        System.out.println("Main da Java2 executando!");
    }

}

public class Java1 {
    public static void main(String[] args) {
        System.out.println("Iniciando Java1...");

        // Chama o main da Java2 diretamente
        Java2.main(new String[]{});

        System.out.println("Java1 continuando após Java2...");
    }

}
```

Saída:

```
Iniciando Java1...
Main da Java2 executando!

Java1 continuando após Java2...
```

Isso funciona porque `main` é um método `static` — pode ser chamado sem instanciar a classe.

Situação 4 — Usando Reflexão (classes desconhecidas em tempo de compilação)

Quando você não conhece o nome da classe em tempo de compilação (por exemplo, o nome vem de um arquivo de configuração ou input do usuário), usa-se reflection com `java.lang.reflect`.

```
import java.lang.reflect.Method;

public class Java1 {

    public static void main(String[] args) {
        try {
            // Carrega a classe pelo nome (como String)
            Class<?> classe = Class.forName("Java2");
```

```
            // Cria uma instância da classe
            Object instancia =
classe.getDeclaredConstructor().newInstance();

            // Obtém o método pelo nome
            Method metodo = classe.getMethod("executar");

            // Invoca o método na instância
            metodo.invoke(instancia);

        } catch (Exception e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }

 }

public class Java2 {
    public void executar() {
        System.out.println("Java2 executada via Reflection!");
    }

 }
```

Quando usar reflection?
Apenas quando realmente necessário — em frameworks, plugins dinâmicos ou sistemas onde a classe não é conhecida em tempo de compilação. Para situações comuns, sempre prefira a instanciação direta com `new`.

Situação 5 — Executar `Java2` como processo separado (nível de sistema operacional)

Essa abordagem cria um processo completamente separado da JVM atual, como se você executasse `java Java2` no terminal. É mais rara, mas possível.

```
import java.io.*;

public class Java1 {

    public static void main(String[] args) throws Exception {

        // Executa Java2 como processo separado do sistema
        ProcessBuilder pb = new ProcessBuilder("java", "Java2");
```

```
        pb.redirectErrorStream(true);

        Process processo = pb.start();

        // Lê a saída do processo Java2
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(processo.getInputStream())
        );

        String linha;
        while ((linha = reader.readLine()) != null) {
            System.out.println("[Java2 diz]: " + linha);
        }

        processo.waitFor();
        System.out.println("Java2 finalizou com código: " +
processo.exitValue());
    }

 }
```

Atenção:
Para isso funcionar, o `Java2.class` precisa estar compilado e no classpath. Use essa abordagem apenas quando precisar de isolamento total entre processos.